

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – Vulnerability Detection

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 3
Weightage:	40% of CIA	Total Marks:	100
CO 3 Statement:	Vulnerability Detection, Severity Analysis, Security Verification Workflow Simulation and Performance Evaluation		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Perform vulnerability detection and classify the identified vulnerabilities based on their severity levels.
- Conduct severity analysis using appropriate metrics such as CVSS and document the impact of each vulnerability.
- Design and simulate a Security Verification Workflow covering detection, remediation, re-testing, and verification.
- Evaluate security and system performance before and after mitigation, and include relevant results, screenshots, and observations.

Question Type	Focus Area	Questions	Marks
Vulnerability Detection	Conceptual Understanding	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Name: B. Tharun Kumar Reddy

Reg no: 192565015

Course code: CSA5803

Course name: DEVSECOPS ENGINEERING AND APPLICATION SECURITY

Vulnerability Detection, Severity Analysis, Security Verification Workflow Simulation and Performance Evaluation

Introduction:

Application security is an important part of modern software development. Applications are connected to the internet, databases, cloud services, APIs, and third-party libraries. Because of this, attackers can exploit weaknesses in applications to steal information, modify data, or interrupt services. Vulnerability detection is the process of identifying these security weaknesses before attackers can use them. In a DevSecOps environment, security testing is integrated into the software development lifecycle. Developers, security teams, and operations teams work together to detect and fix vulnerabilities early. This reduces the cost and effort required to correct security problems after deployment. The major stages considered in this simulation are:

1. Vulnerability Detection
2. Severity Analysis
3. Security Verification
4. Workflow Simulation
5. Performance Evaluation

These stages provide a systematic method for identifying security problems, determining their importance, fixing them, and checking whether the security improvements are successful.

Vulnerability Detection

Vulnerability detection is the first stage of the security testing process. It identifies weaknesses in an application, source code, configuration, dependencies, or running system.

For example, consider an e-commerce application containing the following components:

- User login
- Product search
- Shopping cart
- Payment system
- Customer database
- REST APIs
- Third-party libraries

A security testing tool may identify vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), weak authentication, insecure configuration, or outdated software dependencies.

Types of Vulnerability Detection

Static Analysis

Static Application Security Testing (SAST) analyzes source code without executing the application. It can identify coding problems such as hard-coded passwords, insecure functions, and possible injection vulnerabilities.

Dynamic Analysis

Dynamic Application Security Testing (DAST) tests a running application. It sends different requests to the application and observes its responses. It can detect problems such as XSS, authentication issues, and insecure HTTP configurations.

Software Composition Analysis

Software Composition Analysis (SCA) checks third-party and open-source dependencies. It identifies outdated libraries and known vulnerabilities associated with those components.

Manual Security Testing

Security professionals can manually test application functionality and investigate vulnerabilities that automated tools may not identify correctly.

Severity Analysis

After vulnerabilities are detected, the next step is to determine how serious each vulnerability is. This process is called severity analysis. Severity analysis helps organizations prioritize vulnerabilities so that the most dangerous issues can be fixed first.

Not every vulnerability creates the same level of risk. For example, a vulnerability that allows an attacker to access the complete customer database is much more serious than a missing security header.

Factors Used in Severity Analysis

The following factors can be considered while determining vulnerability severity:

Impact

Impact describes the possible damage caused by a vulnerability.

Examples include:

- Loss of confidential information
- Unauthorized access
- Financial loss
- Data modification
- Service interruption
- Reputation damage

Exploitability

Exploitability describes how easily an attacker can take advantage of a vulnerability.

A vulnerability that can be exploited remotely without authentication generally requires more urgent attention than a vulnerability requiring local access.

Attack Complexity

Attack complexity determines how difficult it is to perform an attack. Simple attacks are generally considered more dangerous because they require fewer technical skills.

Required Privileges

Some vulnerabilities require an attacker to already have an account or administrative privileges. Vulnerabilities that can be exploited without authentication may receive higher priority.

Security Verification Workflow Simulation

After vulnerabilities are identified and prioritized, developers must fix them. However, fixing a vulnerability is not the final step. The application must be tested again to verify that the vulnerability has actually been removed.

Security verification is the process of confirming that security fixes work correctly and do not introduce new problems.

Security Verification Workflow

A basic security verification workflow can be represented as:

Code Development → Security Scan → Vulnerability Detection → Severity Analysis → Fix → Verification Scan → Security Approval → Deployment

Each stage has a specific purpose.

Step 1 – Code Development

Developers create or modify application code. Security requirements should be considered during development rather than waiting until the application is completed.

Step 2 – Security Scan

Automated security tools scan the application. SAST can analyze source code, DAST can test the running application, and SCA can check dependencies.

Step 3 – Vulnerability Detection

The security tools generate findings. Each finding should contain information such as:

- Vulnerability type
- Affected component
- Location
- Severity
- Possible impact
- Recommended solution

Step 4 – Severity Analysis

The security team analyzes the findings and assigns priority. Critical and high-risk vulnerabilities are normally handled before lower-risk findings.

Step 5 – Vulnerability Fix

Developers correct the identified vulnerabilities.

For example, if SQL Injection is identified, developers can use parameterized queries instead of directly combining user input with SQL statements.

Instead of accepting unsafe user input directly, the application should validate and safely process the input.

Step 6 – Verification Scan

After the fix is implemented, the security scan is performed again. The purpose is to confirm that the vulnerability is no longer detected.

For example:

Before Fix:

SQL Injection – Critical – Detected

After Fix:

SQL Injection – Critical – Resolved

Step 7 – Security Approval

If the verification scan confirms that critical vulnerabilities have been resolved, the security team can approve the application for the next stage.

Step 8 – Deployment

After security approval, the application can be deployed to the required environment.

Workflow Simulation and Performance Evaluation

Security Workflow Simulation

Workflow simulation is the process of representing how security activities are performed from vulnerability detection to final verification. It helps demonstrate how a DevSecOps security process operates in a realistic environment.

For this simulation, an online shopping application is considered. The application contains a login page, product search, shopping cart, payment API, and customer database.

Simulated Security Workflow

The workflow begins when a developer submits updated application code.

Developer Commit → Build → Security Testing → Finding Generation → Severity Analysis → Developer Notification → Vulnerability Fix → Re-test → Approval

If the security scan identifies a critical vulnerability, the workflow stops the deployment process until the issue is resolved.

For example, assume the first scan produces:

- 2 Critical vulnerabilities
- 3 High vulnerabilities
- 4 Medium vulnerabilities
- 2 Low vulnerabilities

The two critical vulnerabilities are assigned to developers immediately.

Automated Security Gate

A security gate can be implemented in a CI/CD pipeline.

Example rule:

If Critical vulnerabilities > 0 → Build Failed

If High vulnerabilities > allowed limit → Build Failed

If only Low vulnerabilities exist → Build Allowed

This approach prevents vulnerable software from automatically reaching production.